
Konzeption und Implementierung eines Webservice zum Scheduling der SYNCROTESS BM Transportoptimierung

im Studiengang Angewandte Mathematik und Informatik

von Niklas Paar

Matrikelnummer: 3637975

Diese Arbeit wurde betreut von

1. Prüfer: Prof. Dr. rer. nat. Alexander Voß
2. Prüfer: Frank Fleischhack

Aachen, Dezember 2025

Sperrvermerk

Die vorliegende Seminararbeit enthält vertrauliche Informationen der INFORM GmbH. Sie darf ausschließlich von den Prüfern und den berechtigten Personen der Fachhochschule Aachen eingesehen werden. Eine Weitergabe an Dritte sowie eine Veröffentlichung, Vervielfältigung oder Verwertung der Inhalte ist ohne ausdrückliche Genehmigung der INFORM GmbH nicht gestattet.

Inhaltsverzeichnis

1. Einleitung	5
1.1. Hintergrund und Motivation	5
1.2. Ziel	6
1.3. Vorgehensweise	6
2. Ist-Analyse	8
2.1. SYNCROTESS BM un der Tserv	8
2.2. Optimierer	9
2.2.1. ftlopt und MIXOR	9
2.2.2. coflo	9
2.3. Aktueller Ablauf einer Optimierung	10
2.3.1. Dispatch-Gruppen und Optimierungstypen	10
2.3.2. Bestehendes Scheduling: Mehrere FIFO-Warteschlangen	11
2.4. Probleme und Einschränkungen des Ist-Zustands	11
2.4.1. Starre, hartcodierte Parallelitätsstruktur	12
2.4.2. Kein typenübergreifendes Scheduling und fehlende Priorisierung	12
2.4.3. Fork-Mechanismus	13
2.4.4. Ausschließliche Erreichbarkeit über den Tserv	13
2.4.5. Fehlende Ressourcenkontrolle und Laufzeitüberwachung	13
2.4.6. Zusammenfassung	14
3. Anforderungen an den Scheduler-Webservice	15
3.1. Vorbedingungen	15
3.2. Anwendungsfälle	15
3.3. technische Anforderungen	15
3.4. kurze Beschreibung der Algorithmen	15
3.5. Selektierung der Algorithmen	15
3.6. Zusammenfassung	15
4. Konzeptionierung	16
4.1. Beschreibung der ausgewählten Algorithmen	16
4.2. Analyse der Algorithmen	16

5. Evaluation	17
6. Fazit & Ausblick	18

1. Einleitung

Diese Bachelorarbeit thematisiert die Implementierung und Konzeptionierung eines Webservice zum Scheduling der SYNCROTESS BM Transportoptimierung. Diese Arbeit wird von der INFORM GmbH (Institut für Operations Research und Management GmbH) betreut, einem Softwareunternehmen, das auf Operations Research spezialisiert ist.

1.1. Hintergrund und Motivation

Die von INFORM entwickelte Softwarelösung SYNCROTESS BM richtet sich an Unternehmen aus der Baustoff- bzw. der Logistikbranche und deckt unter anderem die Planung von Transporten ab. Ein Bestandteil dieser Lösung ist die Transportoptimierung, wodurch Dispositionsentscheidungen sinnvoll getroffen werden können. Logistische Abläufe sollen dadurch effizienter und schneller optimiert werden. Damit diese Optimierungsläufe durchgeführt werden können, werden spezialisierte Programme - sogenannte Optimierer - eingesetzt.

Derzeit werden die Optimierer innerhalb des *Tserv* ausgeführt. Dies ist das Backend des SYNCROTESS-Produkts für das Transportmanagement. Der *Tserv* enthält die Businesslogik und führt die einzelnen Optimierer intern aus. Die Optimierer-Prozesse werden momentan als Fork, also als neuer Prozess auf dem selben Host-System, ausgeführt. Dadurch sind die Prozesse durch die Hardware des Host Systems limitiert. Diese kann nur eingeschränkt oder umständlich angepasst werden. Zusammengefasst führt das zu folgenden Einschränkungen:

- Die Optimierungsprozesse werden nur bedingt parallel ausgeführt
- Die Optimierer sind ausschließlich über den *Tserv* ansprechbar
- Geringe Skalierbarkeit
- Keine Priorisierung der Optimierungsprozesse

Insbesondere kann die fehlende Priorisierung und die fehlende Parallelisierung der einzelnen Optimierungsprozesse bei zeitkritischen Szenarien zu Problemen führen, wenn die Optimierung durch weniger relevante Prozesse blockiert wird. Es fehlt daher ein Mechanismus, der die Prozesse priorisiert und parallelisiert abarbeiten kann.

1.2. Ziel

Das Ziel dieser Arbeit ist es, einen Webservice zu konzipieren und zu implementieren. Der Webservice soll die Ausführung der Optimierungsprozesse vom Tserv entkoppeln und über eine REST-Schnittstelle steuern. Dabei sollen mehrere Optimierungsprozesse parallel laufen können. Eingehende Anfragen sollen über eine Warteschlange koordiniert werden; Grenzen wie maximale Laufzeit und maximale Arbeitsspeicherauslastung sollen dabei berücksichtigt werden.

Um die Reihenfolge der abzuarbeitenden Prozesse festlegen zu können, soll eine geeignete Scheduling Strategie gefunden werden. Dazu müssen einzelne Algorithmen untersucht und verglichen werden. Eine Herausforderung dabei ist, dass die Ausführungszeiten der Optimierer im Voraus unbekannt sind und auch nicht ausreichend abgeschätzt werden können. Durch eine Anforderungsanalyse werden verschiedene Scheduling-Verfahren vorgestellt, bewertet und anhand von definierten Kriterien evaluiert.

Die Implementierung einzelner Algorithmen dient zunächst ausschließlich der Evaluierung und ist nicht für den Produktiveinsatz vorgesehen. Stattdessen ist das Ziel, eine fundierte Entscheidungsgrundlage für die Auswahl einer geeigneten Scheduling-Strategie zu schaffen.

1.3. Vorgehensweise

Zunächst soll eine Ist-Analyse des bestehenden Systems durchgeführt werden. Dazu wird der Aufbau des Tservs und speziell die Optimierungsverfahren erläutert.

Darauf aufbauend werden Anforderungen definiert, die der neue Webservice erfüllen muss. Hierbei werden funktionale Anforderungen mithilfe von definierten Anwendungsfällen erarbeitet und technische Randbedingungen festgelegt.

Im Anschluss folgt eine Recherche verschiedener Scheduling-Algorithmen, die hinsichtlich ihrer Eignung bewertet werden. Anschließend werden die Algorithmen,

die gegen Ausschlusskriterien verstoßen, verworfen. Die verbleibenden Verfahren werden weiter auf erarbeitete Kriterien untersucht.

Abschließend wird auf der Basis der Untersuchungen ein Scheduling-Algorithmus für die Implementierung des Webservices festgelegt.

2. Ist-Analyse

Um zu verstehen, warum die Implementierung eines Scheduling-Webservices sinnvoll ist, muss man zunächst verstehen, wie das bestehende System arbeitet. Dieses Kapitel beschreibt nun den Aufbau der aktuellen Implementierung des Tserv, dem zentralen Anwendungsserver der SYNCROTESS-BM-Plattform. Zusätzlich wird der darin beinhaltete Ablauf der Optimierung beschrieben. Abschließend werden die Einschränkungen und Probleme des Ist-Zustands herausgearbeitet, welche den Ausgangspunkt und die Voraussetzung für die Konzeption des OptimSchedulers geben.

2.1. SYNCROTESS BM un der Tserv

SYNCROTESS BM ist eine von der InformGmbH entwickelte Softwarelösung für die Transportplanung in der Logistikbranche, speziell im Bereich der Baustoffe. Kern dieser Plattform ist der Tserv, ein zentraler Anwendungsserver, welcher die Geschäftslogik der Software beinhaltet, bündelt und die Kommunikation zwischen den verschiedenen Systemkomponenten koordiniert.

Dieser Tserv ist eine komplexe und über Jahre gewachsene Serveranwendung, die mit dem Betriebssystem Linux betrieben wird. Seine Aufgabe ist es, Stammdaten und Datenabfragen zu verarbeiten. Darüber hinaus verarbeitet er CRUD-Operationen zu den einzelnen feldern. Zusätzlich steuert er den gesamten Lebenszyklus der Transportvorgänge. Ein wesentlicher Bestandteil ist dabei die Transportoptimierung. Auf Anfrage führt der Tserv spezialisierte Optimierungsalgorithmen aus, welche auf Grundlage der vorgelegten Daten (textcolorredEinfügen was für Daten) optimale Dispositionsentscheidungen bestimmen.

2.2. Optimierer

Die Optimierungslogik ist nicht im Tserv selbst implementiert, sondern in eigenständigen, separaten Programmen - den sogenannten Optimierern. Die Optimierer werden von Tserv zur Laufzeit aufgerufen und berechnen auf Basis einer JSON-Eingabedatei auch eine JSON-Ausgabedatei. Die Optimierer sind dabei in C++ implementiert. Zusätzlich werden externe Bibliotheken, wie *Boost* genutzt und die Optimierer werden mit `Makefile` gebaut. Diese Skripte sind, dann als standalone lauffähig, werden aber ausschließlich durch den Tserv gestartet.

2.2.1. *ftlopt* und MIXOR

ftlopt ist das zentrale Optimierungsframework für die Transportoptimierung. Es optimiert die Zuordnung von Kundenaufträgen zu LKW-Schichten und Werken. Die Basis dafür sind verschiedenste Nebenbedingungen und Zielfunktionen. Ein konkreter Optimierungsansatz für den Transportbeton-Geschäftskontext, den *ftlopt* zur Verfügung stellt, ist der Algorithmus MIXOR (Mixed Operations Research).

MIXOR unterstützt einen Mehrschicht-Betrieb, also sowohl Tag- als auch Nachtschichten. Darüber hinaus unterstützt MIXOR eine dynamische Pausenplanung. Die Ein- und Ausgabe ist jeweils als JSON realisiert. Ein typischer Aufruf sieht wie folgt aus:

```
ftlopt -i ftlopt1.json -w /tmp/workdir/ -s ftlopt1.sol.json
```

In dem Kommando sieht man, dass über `-i` die Eingabedatei übergeben wird. Durch `-w` wird das Arbeitsverzeichnis übergeben, und mit `-s` wird die Ausgabedatei mit dem Optimierungsergebnis definiert.

2.2.2. *coflo*

coflo ist ein weiterer Optimierer, der speziell für die Zementoptimierung im Bereich Full Truck Load (FTL) entwickelt wurde. Full Truck Load bezeichnet Transporte, wobei ein einzelner Auftrag das gesamte Fahrzeug auslastet - im Gegensatz zu Teilladungen, wobei mehrere Aufträge auf einem Fahrzeug gebündelt werden. *coflo* adressiert damit den Transport von einem massengut, wie beispielsweise Zement, während *ftlopt* auf Beton optimiert ist.

Analog zu *ftlopt* wird auch *coflo* vom Tserv über ein Shell-Skript-Template gestartet und kommuniziert dateibasiert über JSON. [\[Konkreten coflo-Aufruf ergänzen\]](#)

2.3. Aktueller Ablauf einer Optimierung

[\[Ggf. Abbildung des aktuellen Ablaufs einfügen\]](#)

Momentan läuft die Optimierung im System wie folgt ab:

- Der Tserv enthält die Optimierungssteuerung als internes Modul
- Wenn der Build-Prozess des Tservs läuft, dann werden die Optimierer-Binaries durch den *make*-Aufruf erzeugt und unter `$INSTRROOT/bin/` abgelegt
- Dort wird dann das passende Shell-Skript-Template instanziiert und ausgeführt, welches den entsprechenden Optimierer-Prozess startet.

Der Optimierer-Prozess wird dabei als *Fork* gestartet - das bedeutet, dass ein Kindprozess auf demselben Host-System erzeugt wird, der die Ausführung des optimierers übernimmt. Dabei wird eine Dateischnittstelle zwischen Tserv und Optimierer verwendet. Der Tserv schreibt also die Eingabedatei im JSON-Format, die der Tserv einliest und weiterverarbeitet.

2.3.1. Dispatch-Gruppen und Optimierungstypen

Eine Dispatch Gruppe repräsentiert eine logische Einheit der Transportplanung in unserer Software. Diese stellt eine Planungsregion aus ein oder mehreren Werken dar. für jede Dispatchgruppe wird unabhängig optimiert, und innerhalb jeder Dispatchgruppe gibt es zusätzlich folgende Optimierungstypen:

- **RT** (Realtime): Dieser Optimierungstyp beschreibt die Echtzeitoptimierung unserer Software. Hier wird auf den direkten Fahrzeugstatus oder Änderungen in der Schichtplanung reagiert, und anhand dessen kann geplant werden.
- **OEV0, OEV1, OEV2**: Dieser Optimierungstyp beschreibt die Varianten der *Optimierten Endvorschau*. Planungsläufe, die den Tagesplan auf Basis aller vorliegenden Aufträge und Ressourcen neu berechnen und dem Disponenten als Entscheidungsgrundlage dienen.

- **PP** (Preplanning): Dieser Optimierungstyp beschreibt die Vorplanung für den kommenden Betriebstag und erstellt auf Basis der prognostizierten Auftragslage Aufträge für die Zukunft.

Die Optimierungstypen sind bei allen Dispatchgruppen identisch. Darüber hinaus wird für jeden dieser Optimierungstypen ein eigener Tserv Fork-Prozess erstellt, sodass RT, OEV0 und PP innerhalb einer Dispatch-Gruppe parallel ausgeführt werden können. Oev1 und Oev2 bilden dabei eine kleine Ausnahme, da diese auf demselben Fork-Prozess laufen und dabei sich immer alternierend mit der Ausführung abwechseln.

2.3.2. **Bestehendes Scheduling: Mehrere FIFO-Warteschlangen**

Pro Optimierungstyp existiert also ein eigener Fork Prozess. Somit existiert, dann auch eine eigene Warteschlange für jedem dieser Typen. Diese wird nach dem *First-In-First-Out*-Prinzip (FIFO) abgearbeitet. Die Anfrage, die zuerst eintrifft wird also zuerst abgearbeitet. In der Praxis tritt es allerdings eigentlich nicht auf, dass die Warteschlange verwendet werden muss, da man in der Software nicht mehrere RT,PP oder OEV Optimierungen anstoßen kann. Ausnahme bilden hierbei OEV1 und OEV2. Bei diesen Prozessen muss der eine auf den anderen warten, wobei alternierend gewechselt wird.

Das bedeutet, dass das bestehende System bereits über eine rudimentäre Form von Parallelität und Scheduling verfügt. Diese ist jedoch starr, da sie ausschließlich auf der festen Struktur der Dispatch-Gruppen und Optimierungstypen basiert.

2.4. **Probleme und Einschränkungen des Ist-Zustands**

Der beschriebene Ablauf des Optimierungsablaufs ist funktional und verfügt bereits über eine rudimentäre Form von Scheduling und Parallelität. Das Problem ist, dass der Ablauf dennoch strukturelle Einschränkungen aufweist, die mit steigenden Anforderungen an die Plattform problematisch werden können. So erfordert etwa die Anbindung eines neuen Optimierers oder die Einführung eines zusätzlichen Optimierungstyps stets Änderungen am Tserv-Quellcode selbst. Ebenso wächst

mit steigender Anzahl von Dispatch-Gruppen und gleichzeitigen Optimierungsanfragen der Druck auf die Hardware des Host-Systems, da alle Prozesse denselben Rechner teilen. Auf der anderen Seite steigen auch die Anforderungen an die externe Nutzbarkeit der Optimierer: Szenarien, in denen externe Systeme oder Services die Optimierung direkt anstoßen sollen, lassen sich mit der bestehenden Architektur nicht abbilden. Die folgenden Abschnitte beschreiben die einzelnen Einschränkungen im Detail.

2.4.1. Starre, hartcodierte Parallelitätsstruktur

Die Parallelität im derzeitigen System ist fest an die Struktur der Dispatch-Gruppen und der verschiedenen Optimierungstypen (RT, OEVO, ...) gebunden. Es läuft für jeden Typen ein Prozess parallel, dies passt sich nicht an die Anforderungen an. Eine Anpassung dieser Struktur, beispielsweise um zusätzliche Parallelität unabhängig von Dispatchgruppen oder Optimierungstypen würde die Struktur auflockern und das hinzufügen neuer Optimierungstypen vereinfachen. Dadurch würde auch die Parallelität eine Eigenschaft der Systemarchitektur werden und kein konfiguriertes Verhalten sein.

2.4.2. Kein typenübergreifendes Scheduling und fehlende Priorisierung

Jede Warteschlange ist von der jeweils anderen vollständig isoliert. Zudem werden die Warteschlangen strikt nach FIFO abgearbeitet. Eine Priorisierung über eine Typgrenze hinweg ist daher unmöglich. Beispiel: Eine Priorisierung über Typgrenzen hinweg ist nicht möglich: Eine dringende RT-Optimierung kann nicht vorgezogen werden, wenn in ihrer Warteschlange bereits ein anderer RT-Lauf wartet – und sie kann erst recht nicht Ressourcen beanspruchen, die gerade ein OEVO- oder PP-Prozess belegt. **Bild einfügen**

Umgekehrt kann ein weniger wichtiger Optimierungstyp keine Ressourcen für einen wichtigeren Typen freigeben. Somit können weniger wichtigere PP-Prozesse auch wichtigere RT-Prozesse überholen. **Bild einfügen**

Durch das Fehlen einer übergeordneten Scheduling-Logik kann es in zeitkritischen Szenarien zu unnötigen Verzögerungen kommen.

2.4.3. Fork-Mechanismus

Dadurch, dass alle Optimierer-Prozesse als Fork auf demselben Host-System gestartet werden, teilen sie sich auch die Ressourcen des Systems, also auch Arbeitsspeicher und CPU-Kerne. Die Rechenkapazität ist damit direkt an die Hardware des Host-Systems gebunden, die sich nur eingeschränkt oder mit hohem Aufwand anpassen lässt. Eine flexiblere Skalierung wäre also wünschenswert.

Ein weiteres Problem welches durch den Fork-Mechanismus entsteht ist, dass immer ein gesamter Tserv für eigentlich nur einen kleinen Teil des Tservs geforkt wird. Dadurch wird zusätzlich unnötig Arbeitsspeicher verbraucht.

2.4.4. Ausschließliche Erreichbarkeit über den Tserv

Die Optimierung ist ausschließlich über den Tserv ansprechbar, obwohl die Programme bzw. die Skripte technisch gesehen bereits jetzt als standalone, also unabhängig, lauffähig sind. Dennoch sind sie im derzeitigen System in den Tserv eingebunden. Eine unabhängige Nutzung, beispielsweise für eine externe Anbindung oder den Betrieb in anderer Infrastruktur ist nicht möglich ohne auch den Tserv zu nutzen.

2.4.5. Fehlende Ressourcenkontrolle und Laufzeitüberwachung

Derzeit existiert keine Überwachung der Systemressourcen bei der Optimierungsausführung. Bei fehlerhaften oder lang laufenden Optimierungen gibt es keinen automatischen Abbruchmechanismus. Zudem wird vor dem start eines Optimierers nicht geprüft, ob genug Arbeitsspeicher für weitere Prozesse vorhanden ist. Dies kann bei hoher Auslastung zu Instabilität des Gesamtsystems führen.

2.4.6. Zusammenfassung

Tabelle 2.1 fasst die identifizierten Einschränkungen des Ist-Zustands zusammen und stellt ihnen die jeweils angestrebte Verbesserung durch den OptimScheduler gegenüber.

Einschränkung (Ist)	Ziel (Soll)
Parallelität starr an Dispatch-Gruppen und Optimierungstypen gebunden; nicht konfigurierbar	Flexibel konfigurierbare Parallelität, unabhängig von fixer Typstruktur
Warteschlangen isoliert und strikt FIFO; kein typenübergreifendes Scheduling – weniger wichtige PP-Prozesse können RT-Prozesse überholen	Priorisierungsfähiges Scheduling über alle Optimierungstypen hinweg
Fork-Mechanismus: Rechenkapazität an Host-Hardware gebunden; Fork des gesamten Tserv verursacht unnötigen Speicherverbrauch	Entkopplung vom Tserv; flexible Skalierbarkeit und gezielter Ressourceneinsatz
Optimierer ausschließlich über den Tserv erreichbar, obwohl technisch standalone lauffähig	Eigenständiger Webservice mit REST-Schnittstelle
Keine Ressourcenüberwachung; kein automatischer Abbruch bei fehlerhaften Optimierungen	Timeout-Handling und Überwachung der Systemauslastung

Tabelle 2.1.: Einschränkungen des Ist-Zustands und angestrebte Verbesserungen

Diese Defizite bilden die fachliche Grundlage für die Anforderungsanalyse im folgenden Kapitel.

3. Anforderungen an den Scheduler-Webservice

3.1. Vorbedingungen

3.2. Anwendungsfälle

3.3. technische Anforderungen

3.4. kurze Beschreibung der Algorithmen

3.5. Selektierung der Algorithmen

3.6. Zusammenfassung

4. Konzeptionierung

4.1. Beschreibung der ausgewählten Algorithmen

4.2. Analyse der Algorithmen

5. Evaluation

6. Fazit & Ausblick

Eidesstattliche Erklärung

Ich versichere hiermit an Eides statt, dass ich die vorliegende Arbeit selbstständig und ohne Benutzung anderer als der angegebenen Hilfsmittel angefertigt habe. Alle Stellen, die wörtlich oder sinngemäß aus Veröffentlichungen entnommen wurden, sind als solche kenntlich gemacht.

Im Rahmen der Erstellung dieser Arbeit wurde das KI-System 'KI.connect.nrw' unterstützend zur sprachlichen Überarbeitung sowie zur fachlichen Reflexion und Präzisierung eigenständig entwickelter Argumente genutzt. Eine Übernahme von KI-generierten Texten oder inhaltlichen Lösungsvorschlägen erfolgte nicht. Sämtliche fachlichen Aussagen, Bewertungen und Schlussfolgerungen wurden eigenständig erarbeitet und verantwortet. Die Nutzung erfolgte im Einklang mit der Zweckbestimmung des Systems sowie unter Beachtung datenschutz- und urheberrechtlicher Vorgaben.

Ort, Datum: _____ Unterschrift: _____